

OSSP PRACTICAL WEEK 5

TASK 1:

An **anonymous pipe** is an inter-process communication (IPC) mechanism used to transfer data between related processes, usually a parent and child process. In this program, the **parent process acts as the producer** and generates data, while the **child process acts as the consumer** and reads the data from the pipe. The pipe() system call creates the communication channel, fork() creates the child process, write() sends data through the pipe, and read() receives it. The unused ends of the pipe are closed to ensure proper communication. The communication time can be measured to analyze the efficiency of data transfer between the processes.

```

GNU nano 7.2 producer_consumer.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
#include <time.h>

int main()
{
    int pipefd[2];
    pid_t pid;

    char message[] = "Hello from Producer! This data is sent through an anonymous pipe.";
    char buffer[100];

    struct timespec start, end;

    // Create pipe
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    // Start measuring time
    clock_gettime(CLOCK_MONOTONIC, &start);

    // Create child process
    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid > 0)
    {
        // Parent = Producer
        close(pipefd[0]); // Close reading end
        printf("Producer: Sending data...\n");
        write(pipefd[1], message, strlen(message) + 1);
    }

    if (pid == 0)
    {
        // Child = Consumer
        close(pipefd[1]); // Close writing end
        read(pipefd[0], buffer, sizeof(buffer));
        printf("Consumer: Received data: %s\n", buffer);
        printf("Communication completed.\n");
        clock_gettime(CLOCK_MONOTONIC, &end);
        printf("Communication time: %f seconds\n", end.tv_sec + end.tv_nsec / 1000000000.0);
    }

    wait(&pid);
    return 0;
}

```

```

root@LAPTOP-0PA61QEN:~# ls /home
alurusaranya
root@LAPTOP-0PA61QEN:~# su alurusaranya
alurusaranya@LAPTOP-0PA61QEN:/root$ ls
ls: cannot open directory '.': Permission denied
alurusaranya@LAPTOP-0PA61QEN:/root$ cd ~
alurusaranya@LAPTOP-0PA61QEN:~$ pwd
/home/alurusaranya
alurusaranya@LAPTOP-0PA61QEN:~$ nano producer_consumer.c
alurusaranya@LAPTOP-0PA61QEN:~$ gcc producer_consumer.c -o producer_consumer
alurusaranya@LAPTOP-0PA61QEN:~$ ./producer_consumer
Producer: Sending data...
Consumer: Received data:
Hello from Producer! This data is sent through an anonymous pipe.
Communication completed.
Communication time: 0.000686185 seconds

```

```

GNU nano 7.2 producer_consumer.c
    perror("%s\n", strerror(errno));
    exit(EXIT_FAILURE);
}

if (pid > 0)
{
    // Parent = Producer
    close(pipefd[0]); // Close reading end
    printf("Producer: Sending data...\n");
    write(pipefd[1], message, strlen(message) + 1);
    close(pipefd[1]); // Close writing end
    wait(NULL);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double time_taken =
        (end.tv_sec - start.tv_sec +
         end.tv_nsec - start.tv_nsec) / 1e9;
    printf("Communication completed.\n");
    printf("Communication time: %.9f seconds\n", time_taken);
}
else
{
    // Child = Consumer
    close(pipefd[1]); // Close writing end
    read(pipefd[0], buffer, sizeof(buffer));
    printf("Consumer: Received data:\n");
    printf("%s\n", buffer);
    close(pipefd[0]); // Close reading end
    exit(EXIT_SUCCESS);
}

return 0;
}

```

TASK 2:

The shell command `ls -l | grep ". "` demonstrates **inter-process communication using a pipe**, where the output of `ls -l` is passed as input to `grep ". "`. In this program, two child processes are created using `fork()`. The first child executes `ls -l` using `exec()` and redirects its standard output to the pipe using `dup2()`. The second child redirects its standard input from the pipe and executes `grep ". "` using `exec()`. The `pipe()` system call provides the communication channel between the two processes, while `dup2()` connects the pipe to standard input/output, thereby reproducing the behavior of the shell pipeline.

2520080012

```
alurusaranya@LAPTOP-0PA61QEN:~$ nano producer_consumer.c
alurusaranya@LAPTOP-0PA61QEN:~$ nano ls_grep.c
alurusaranya@LAPTOP-0PA61QEN:~$ gcc ls_grep.c -o ls_grep
alurusaranya@LAPTOP-0PA61QEN:~$ ./ls_grep
total 340
-rw-rw-r-- 1 alurusaranya alurusaranya 22 Aug 12 06:54 File1.txt
-rw-rw-r-- 1 alurusaranya alurusaranya 22 Aug 12 06:58 File2.txt
drwxrwxr-x 11 alurusaranya alurusaranya 4096 Aug 3 05:28 OSSP
-rwxrwxr-x 1 alurusaranya alurusaranya 16400 Aug 3 05:55 fork_exec
-rw-rw-r-- 1 alurusaranya alurusaranya 946 Aug 3 05:55 fork_exec.c
-rwxrwxr-x 1 alurusaranya alurusaranya 15960 Aug 3 05:05 hello
-rw-rw-r-- 1 alurusaranya alurusaranya 80 Aug 3 05:05 hello.c
-rwxrwxr-x 1 alurusaranya alurusaranya 16352 Aug 29 08:20 ls_grep
-rw-rw-r-- 1 alurusaranya alurusaranya 1491 Aug 29 08:20 ls_grep.c
-rwxrwxr-x 1 alurusaranya alurusaranya 16880 Aug 24 03:48 myshell
-rwxrwxr-x 1 alurusaranya alurusaranya 16304 Aug 24 04:52 process_fork
-rw-rw-r-- 1 alurusaranya alurusaranya 994 Aug 24 04:53 process_fork.c
-rwxrwxr-x 1 alurusaranya alurusaranya 16488 Aug 29 08:16 producer_consumer
-rw-rw-r-- 1 alurusaranya alurusaranya 1510 Aug 29 08:20 producer_consumer.c
-rw-rw-r-- 1 alurusaranya alurusaranya 3238 Aug 24 03:48 shell.c
-rwxrwxr-x 1 alurusaranya alurusaranya 16224 Aug 12 07:04 task1
-rw-rw-r-- 1 alurusaranya alurusaranya 397 Aug 12 07:03 task1.c
-rwxrwxr-x 1 alurusaranya alurusaranya 16488 Aug 24 05:08 task1_shell
-rw-rw-r-- 1 alurusaranya alurusaranya 1385 Aug 24 05:10 task1_shell.c
-rwxrwxr-x 1 alurusaranya alurusaranya 16224 Aug 12 06:58 task2
-rw-rw-r-- 1 alurusaranya alurusaranya 727 Aug 12 06:57 task2.c
-rwxrwxr-x 1 alurusaranya alurusaranya 16448 Aug 24 05:12 task2_shell
-rw-rw-r-- 1 alurusaranya alurusaranya 2383 Aug 24 05:11 task2_shell.c
-rwxrwxr-x 1 alurusaranya alurusaranya 16280 Aug 12 07:06 task3
-rw-rw-r-- 1 alurusaranya alurusaranya 1001 Aug 12 07:05 task3.c
-rwxrwxr-x 1 alurusaranya alurusaranya 16272 Aug 12 07:07 task4
-rw-rw-r-- 1 alurusaranya alurusaranya 1092 Aug 12 07:07 task4.c
-rwxrwxr-x 1 alurusaranya alurusaranya 19424 Aug 12 08:28 task5
-rw-rw-r-- 1 alurusaranya alurusaranya 1115 Aug 12 08:28 task5.c
-rwxrwxr-x 1 alurusaranya alurusaranya 16352 Aug 24 03:11 week4_t1
-rw-rw-r-- 1 alurusaranya alurusaranya 1210 Aug 24 03:10 week4_t1.c
```

```
GNU nano 7.2 ls_grep.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    // Create pipe
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    // Create first child
    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid1 == 0)
    {
        // First child: execute ls -l
        close(pipefd[0]); // Close unused read end

        // Redirect stdout to pipe
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        // Execute ls -l
        execlp("ls", "ls", "-l", NULL);

        // If exec fails
        perror("execlp ls");
        exit(EXIT_FAILURE);
    }
}
```

2520080012

```
// Create second child
pid2 = fork();

if (pid2 < 0)
{
    perror("fork");
    exit(EXIT_FAILURE);
}

if (pid2 == 0)
{
    // Second child: execute grep "."
    close(pipefd[1]); // Close unused write end

    // Redirect stdin from pipe
    dup2(pipefd[0], STDIN_FILENO);

    close(pipefd[0]);

    // Execute grep "."
    execlp("grep", "grep", ".", NULL);

    // If exec fails
    perror("execlp grep");
    exit(EXIT_FAILURE);
}

// Parent process
close(pipefd[0]);
close(pipefd[1]);

// Wait for both children
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("Command executed successfully.\n");

return 0;
}
```